

Taller 4 - IA

Nombre	Código
Daniel Vergara	202320392
Mariana Rodríguez	202421258
Martín de Angulo	202421628
Yara Gutiérrez	202511181

Punto 6A: Análisis de los algoritmos

En este taller se implementó un sistema de planificación automatizada para la “Operación Fénix”. El problema consiste en que un robot SAR debe moverse en una cuadrícula, recoger suministros médicos, preparar un puesto médico y rescatar uno o varios pacientes. Para esto se usaron acciones tipo PDDL, búsqueda hacia adelante, búsqueda hacia atrás, A* con heurísticas y planificación jerárquica HTN.

Para el análisis se usan las siguientes variables:

Variable	Significado
n	Número de fluentes posibles del problema
a	Número de acciones instanciadas posibles
d	Profundidad del plan solución
b	Factor de ramificación, con $b \leq a$
S	Número de estados alcanzables, con $S \leq 2^n$

Punto 1: Modelado del dominio PDDL

En este punto se definieron las acciones del dominio: Move, PickUp, PutDown, SetupSupplies y Rescue. También se

implementaron funciones para verificar si una acción es aplicable y para calcular el estado resultante después de ejecutarla.

Función	Tiempo	Espacio	Explicación
is_applicable	$O(n)$	$O(1)$	Revisa si las precondiciones positivas están en el estado y si las negativas no están.
apply_action	$O(n)$	$O(n)$	Crea un nuevo estado quitando los fluentes del delete list y agregando los del add list.
get_applicable_actions	$O(an)$	$O(a)$	Revisa todas las acciones instanciadas y filtra las que son aplicables.

Estas funciones no son planificadores por sí solas, pero son la base de todos los algoritmos posteriores. Si estas funciones fallan, los planes generados podrían contener acciones inválidas o estados incorrectos.

Punto 2: Forward Planning con BFS

El planificador forwardBFS parte del estado inicial y expande estados aplicando acciones válidas hasta encontrar un estado que satisfaga la meta.

Criterio	Análisis
Tiempo	$O(b^d)$ en el caso normal por profundidad, o $O(a \cdot S)$ si se considera todo el espacio de estados alcanzable.
Espacio	$O(b^d)$, porque BFS guarda la frontera y los estados visitados.
Complejidad	Sí, si el espacio de estados es finito.
Optimalidad	Sí, si todas las acciones tienen el mismo costo, como ocurre en este proyecto.

BFS es fácil de entender y garantiza encontrar el plan más corto en número de acciones. Sin embargo, puede volverse costoso en mapas grandes porque explora muchos estados sin usar información del objetivo para guiarse.

Punto 3: Backward Planning

Criterio	Análisis
Tiempo	$O(b^d)$, donde b es el número de acciones relevantes para las metas actuales. En el peor caso puede acercarse a $O(a^d)$.
Espacio	$O(b^d)$, por la frontera y los objetivos visitados.
Complejidad	Teóricamente sí, si no se imponen límites artificiales y se explora todo el espacio de subobjetivos. En nuestra implementación se usa un límite para evitar ejecuciones demasiado largas, por lo que en la práctica no siempre es completo.
Optimalidad	Sí en teoría si se usa BFS sobre regresiones y todas las acciones tienen costo 1. En la práctica depende de que no se corte la búsqueda antes de encontrar solución.

La búsqueda regresiva empieza desde la meta y busca qué condiciones anteriores permitirían alcanzarla. Usa la función regress, que aplica la fórmula:

$$\text{REGRESS}(g, a) = (g - \text{ADD}(a)) \cup \text{PRECOND}(a)$$

En este dominio, backward planning puede ser útil porque se concentra en acciones relevantes para lograr `Rescued(patient)`. Sin embargo, también puede generar muchas combinaciones de subobjetivos, especialmente cuando el mapa es más grande.

Punto 4: A* y heurísticas

El planificador aStarPlanner usa la fórmula:

$$f(n) = g(n) + h(n)$$

donde $g(n)$ es el costo acumulado y $h(n)$ es una estimación del costo restante.

Criterio	Análisis
Tiempo	En el peor caso $O(b^d)$, igual que BFS si la heurística no ayuda.
Espacio	$O(b^d)$, porque A* también guarda frontera y mejores costos vistos.
Complejidad	Sí, si los costos son positivos y el espacio es finito.
Optimalidad	Sí si la heurística es admisible y consistente.

Heurística de ignorar precondiciones

Esta heurística estima cuántas acciones serían necesarias si se ignoraran las precondiciones. En teoría, relajar el problema así debería producir una cota inferior del costo real, porque se está resolviendo una versión más fácil del problema.

Criterio	Análisis
Admisibilidad	En teoría sí, si se calcula el costo óptimo del problema relajado. En nuestra implementación se usa una selección greedy, así que en casos generales no se puede garantizar formalmente.
Consistencia	No se garantiza formalmente con la versión greedy.
Ventaja	Es simple y rápida en problemas pequeños.
Desventaja	Puede ser poco informativa, porque ignora demasiadas restricciones importantes.

Heurística de ignorar delete lists

Esta heurística elimina los efectos negativos de las acciones. Así, los flujos nunca se pierden y el problema se vuelve monótono.

Criterio	Análisis
Admisibilidad	En teoría, la relajación ignore-delete puede ser admisible si se calcula el costo óptimo relajado. En nuestra implementación se usa hill-climbing greedy, por lo que no se garantiza formalmente en todos los casos.
Consistencia	No está garantizada formalmente.
Ventaja	Fue más informativa que ignorar precondiciones en tinyBase, porque redujo los estados expandidos.
Desventaja	Es costosa, porque recalcula muchas acciones aplicables durante la estimación.

Punto 5: Planificación HTN

La planificación HTN usa acciones de alto nivel, como FullRescueMission, PrepareSupplies y ExtractPatient, que se refinan en acciones primitivas.

Criterio	Análisis
Tiempo	En general $O(k^h)$, donde k es el número de refinamientos posibles y h la profundidad de refinamiento.
Espacio	$O(k^h)$, por la frontera de planes parciales.
Completitud	Solo es completa con respecto a la jerarquía definida. Si la jerarquía no contiene un refinamiento válido, no encontrará solución aunque exista un plan clásico.
Optimalidad	No necesariamente. HTN depende del orden y de los refinamientos definidos. Puede

	generar planes buenos, pero no garantiza el plan mínimo global.
--	---

La principal ventaja de HTN es que reduce el espacio de búsqueda. En lugar de probar cualquier combinación de acciones primitivas, el algoritmo sigue una estructura de tareas de alto nivel. Esto lo hace mucho más eficiente en los layouts probados, aunque menos general que BFS o A*.

Comparación

Las siguientes pruebas se ejecutaron en modo silencioso usando -q.

Layout tinyBase

Algoritmo	Heurística	Tiempo	Estados expandidos	Longitud del plan	Resultado
Forward BFS	Ninguna	0.029s	225	9	Exitoso
Backward Search	Ninguna	1.179s	0	9	Exitoso
A*	Ignore Preconditions	1.742s	252	9	Exitoso
A*	Ignore Delete Lists	1.070s	138	9	Exitoso
HTN	No aplica	0.000s	0	9	Exitoso

En tinyBase, todos los algoritmos encontraron un plan de 9 acciones. A* con ignoreDeleteLists expandió menos estados que BFS, lo cual muestra que esta heurística fue más informativa en este caso. Sin embargo, tomó más tiempo que BFS por el costo de calcular la heurística.

Layout smallRescue

Algoritmo	Heurística	Tiempo	Estados expandidos	Longitud del plan	Resultado
Forward BFS	Ninguna	0.661s	1181	23	Exitoso
Backward Search	Ninguna	17.241s	0	—	No encontró plan
A*	ignorePreconditions	66,473s	1316	23	Exitoso
A*	Ignore Delete Lists	44.265s	960	23	Exitoso

En smallRescue, BFS fue más rápido que A*, aunque expandió más estados. Esto ocurre porque la heurística ignoreDeleteLists reduce expansiones, pero es costosa de calcular. Backward Search no encontró solución dentro del límite de subobjetivos, lo que muestra que la búsqueda regresiva puede complicarse cuando el mapa crece.

Layouts HTN

Layout	Problema	Tiempo	Estados expandidos	Longitud del plan	Resultado
tinyHTN	SimpleRescueProblem	0.000s	0	9	Exitoso
htnBase	SimpleRescueProblem	0.000s	0	12	Exitoso
tinyMulti	MultiRescueProblem	0.000s	0	21	Exitoso
smallMulti	MultiRescueProblem	0,001s	0	41	Exitoso

HTN fue el enfoque más rápido en las pruebas realizadas, porque usa una estructura de misión ya definida. En vez de explorar libremente todas las acciones, descompone el problema en pasos más claros: preparar suministros, mover paciente al puesto médico y rescatarlo.

Conclusión

En general, BFS fue el algoritmo más confiable para encontrar planes óptimos en los layouts simples, aunque puede crecer mucho en costo cuando aumenta el espacio de estados. Backward Search es conceptualmente útil porque trabaja desde la meta, pero en este dominio puede generar muchos subobjetivos y fallar en mapas más grandes. A* puede reducir estados expandidos si la heurística es buena, pero las heurísticas implementadas tienen un costo alto de cálculo. Finalmente, HTN fue el método más eficiente en los casos probados, porque aprovecha conocimiento del dominio para reducir la búsqueda, aunque no garantiza encontrar el plan óptimo global ni resolver problemas fuera de la jerarquía definida.

Punto 6B: Reflexión sobre uso de la IA

Durante el desarrollo del taller usamos inteligencia artificial como apoyo para revisar, corregir y mejorar la solución. La IA fue especialmente útil para encontrar errores de ejecución sin tener que hacer todo el proceso de debug manualmente. Por ejemplo, ayudó a identificar problemas en algunos algoritmos, revisar por qué ciertas pruebas no corrían correctamente y proponer correcciones más claras.

También fue útil para entender mejor la sintaxis y la estructura del código. Al ver las correcciones explicadas paso a paso, fue más fácil entender por qué una implementación funcionaba o por qué otra podía generar errores. Esto ayuda no solo a corregir el código, sino también a aprender mejor cómo se relacionan las estructuras de datos, las funciones y los algoritmos de planificación.

Otro aspecto importante fue el análisis de rendimiento. La IA ayudó a identificar que algunos algoritmos podían funcionar correctamente en mapas pequeños, pero volverse lentos en mapas más grandes. Esto permitió entender mejor las diferencias entre BFS, búsqueda regresiva, A* y HTN, no solo desde la teoría sino también desde la ejecución práctica.

Además, la IA fue útil para estructurar el documento final y mejorar la redacción del análisis. Ayudó a organizar las ideas, construir tablas comparativas y expresar los resultados de manera más clara y ordenada. También fue muy útil para plasmar información relevante en el README del proyecto, ya sea de la estructura misma o de pruebas, recomendaciones o limitaciones que iba encontrando cada miembro del grupo.

En general, consideramos que la inteligencia artificial puede acelerar el proceso de aprendizaje si se usa correctamente. No debe reemplazar el trabajo propio ni la

comprensión del código, pero sí puede servir como una herramienta para recibir explicaciones detalladas y personalizadas, corregir errores y mejorar la forma en que se presenta una solución. En este taller, el apoyo de la IA permitió avanzar más rápido y entender mejor los problemas que aparecieron durante la implementación.